

Student Mental Health Monitoring System

Laboratory-10: Test Plan

Field	Details
Prepared By	Mokshi Shah / 241071068 Vanshika Shah / 241071069
Course / Program	Software Engineering / SY BTech CE
Institution	VJTI, Mumbai
Date	DD/Month/2026
Document Version	1.0
Reference Standard	IEEE 829-2008 · 11.5.1 Planning Testing

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope of Testing	3
1.3	Objectives	3
1.4	Intended Audience	4
2	Relationship to Other Documents	4
2.1	Applicable Standards and References	5
3	System Overview	6
3.1	System Architecture	6
3.2	Technology Stack	6
3.3	Database Schema — Entity Relationship Summary	7
3.4	Subsystem Decomposition	8
3.5	Counselling Session State Machine	9
4	Features to be Tested / Not to be Tested	10
4.1	Features to be Tested	10
4.1.1	Subsystem 1 — Authentication & Access Control	10
4.1.2	Subsystem 2 — Screening & Risk Assessment	10
4.1.3	Subsystem 3 — Mood Logging & Profile Management	11
4.1.4	Subsystem 4 — Counselling Session & Audit Management	12
4.2	Features Not to be Tested	13
5	Pass/Fail Criteria	14
5.1	Individual Test Case Criteria	14
5.2	Subsystem-Level Pass Criteria	14
5.3	Overall Plan Pass Criteria	14
5.4	Defect Severity Classification	15
6	Approach	15

6.1	Strategy Overview	15
6.2	Black-Box Techniques	15
6.3	White-Box Techniques	16
6.4	Testing Levels	16
6.5	Test Environment	17
6.6	Seed Test Data	17
6.7	Execution Methodology	17
7	Suspension and Resumption Criteria	18
7.1	Suspension Criteria	18
7.2	Resumption Criteria	18
8	Testing Materials (Hardware/Software Requirements)	19
8.1	Hardware Requirements	19
8.2	Software Requirements	19
8.3	Environment Setup Commands	19
9	Test Cases	20
9.1	Subsystem 1: Authentication & Access Control	20
9.2	Subsystem 2: Screening & Risk Assessment	22
9.3	Subsystem 3: Mood Logging & Profile Management	26
9.4	Subsystem 4: Counselling Session & Audit Management	28
10	Testing Schedule	33
10.1	Overview	33
10.2	Detailed Schedule	33
10.3	Testing Timeline (7-Day View)	35
10.4	Test Case Summary	35
10.5	Testing Responsibilities	35

1. Introduction

1.1 Purpose

The purpose of this Test Plan is to define a structured, repeatable, and academically rigorous testing strategy for the **Student Mental Health Monitoring System (SMHMS)**. This document prescribes the complete verification and validation methodology—encompassing Unit Testing, Integration Testing, and System Testing—to be executed across four critical subsystems of the application architecture.

The test plan employs both **Black-Box** (functional, specification-driven) and **White-Box** (structural, implementation-driven) testing techniques to ensure exhaustive coverage of the system's behavioural and architectural correctness. The overarching objective is to guarantee that the multi-role platform (Student Portal, Counsellor Portal, Admin Dashboard) satisfies all functional requirements, enforces strict **Role-Based Access Control (RBAC)** policies, maintains referential integrity across the MySQL persistence layer, upholds the deterministic state-transition logic governing screening and counselling session lifecycles, and preserves privacy-by-design requirements including AES-256 session note encryption and tamper-evident audit logging.

1.2 Scope of Testing

This test plan targets the following four subsystems, selected for their criticality within the system's operational taxonomy:

light #	Subsystem	Scope of Testing
1	Authentication & Access Control	Email/bcrypt login flow, account lockout after 5 failures, session management via <code>express-session</code> , RBAC middleware enforcement across <code>student</code> , <code>counsellor</code> , and <code>admin</code> route namespaces, 2FA flag handling.
2	Screening & Risk Assessment	PHQ-9/GAD-7/PSS score submission with Joi validation, <code>riskEngine.js</code> composite score computation, atomic UPSERT of <code>risk_profiles</code> , High-risk alert triggering, and screening history retrieval.
3	Mood Logging & Profile Management	Daily mood log submission with one-per-day UNIQUE constraint enforcement, mood history retrieval, student risk profile fetch, and student academic data persistence.
4	Counselling Session & Audit Management	Session request creation, state-guard transitions (<code>Requested</code> → <code>Confirmed</code> → <code>Completed</code>), AES-256 encrypted session notes, officer queue retrieval, audit log insertion and tamper-evident hash chain verification.

1.3 Objectives

The specific testing objectives are formulated as follows:

1. **Functional Correctness:** Validate that every API endpoint and UI interaction produces deterministic, specification-compliant outputs for all valid and invalid input vectors.
2. **Security Enforcement:** Confirm that the RBAC middleware pipeline rejects unauthorised cross-role access attempts with appropriate HTTP 401 `Unauthorized` responses; verify that account lockout activates after five consecutive failed login attempts.
3. **Data Integrity:** Verify that all INSERT, UPDATE, and SELECT operations against the MySQL `smhms_db` database maintain strict referential integrity via enforced FOREIGN KEY and UNIQUE constraints.
4. **State Machine Compliance:** Ensure that counselling session status transitions strictly follow the defined set `{Requested, Confirmed, Completed, Cancelled}` and that each transition is atomically logged in the `audit_logs` table with a valid hash chain entry.
5. **Privacy-by-Design Verification:** Confirm that session notes are stored as AES-256-CBC ciphertext in the database, that guardian data access is gated by `consent_records`, and that audit log entries are tamper-evident via the SHA-256 hash chain trigger.
6. **Regression Safety:** Establish a baseline suite of test cases that can be re-executed after future code modifications to detect unintended behavioural regressions.

1.4 Intended Audience

This document is intended for:

- **Course Instructor / Lab Evaluator** — For academic assessment of testing methodology and coverage.
- **Development Team Members** — For executing the prescribed test procedures and recording empirical evidence.
- **Quality Assurance Reviewers** — For evaluating the completeness and traceability of the test design against defined requirements.

2. Relationship to Other Documents

This Test Plan does not exist in isolation; it derives its requirements, architectural context, and validation criteria from a constellation of interrelated project artefacts. The following table establishes the formal traceability matrix between this document and its source references.

lightgray Document	Relationship	Description
Lab-07 Object Model & CRC Cards	Primary Design Reference	Provides the eleven entity classes, four control classes, twenty-two named associations, and state-machine diagrams. All test cases in this plan are derived from responsibilities and collaborators articulated therein.
Lab-09 Design Transformation & Implementation Framework	Implementation Reference	Documents the mapping of CRC cards to MySQL DDL, associations to FK constraints and JS arrays, contracts to HTTP codes and validators, and design patterns (Observer, Strategy, Facade, Template Method). White-Box test cases directly reference these mappings.
MySQL DDL (smhms_schema.sql)	Data Layer Reference	Defines the exact table structures (users, screening_records, mood_logs, risk_profiles, counseling_sessions, audit_logs, notification_logs, consent_records), UNIQUE constraints, FOREIGN KEY rules, and the tamper-evidence trigger.
Backend Source Code (authRoutes.js, screeningRoutes.js, riskEngine.js, etc.)	Implementation Reference	The actual Express.js route handlers, Passport.js local strategy, Joi validators, and riskEngine scoring logic against which White-Box test cases are structurally designed and verified.
Frontend Source Code (frontend/src/pages/*)	UI Reference	The React.js component hierarchy and client-side routing against which Black-Box system-level test cases validate user-facing workflows and visual outputs.

2.1 Applicable Standards and References

lightgray Standard	Applicability
IEEE 829-2008	Standard for Software and System Test Documentation — informs the structural organisation of this plan.
IEEE 730-2014	Software Quality Assurance Processes — guides the quality criteria definitions in Section 5.
ISTQB Foundation Level Syllabus	Provides the theoretical framework for Black-Box (Equivalence Partitioning, Boundary Value Analysis) and White-Box (Statement Coverage, Branch Coverage) techniques used in Section 6.

lightgray Standard	Applicability
OWASP ASVS v4.0	Application Security Verification Standard — referenced for authentication, session management, and access-control validation requirements.
HIPAA / DISHA (applicable privacy norms)	Privacy-by-design requirements for mental health data; referenced for consent-gate and encryption test cases.

3. System Overview

3.1 System Architecture

The SMHMS is architected as a decoupled, full-stack web application adhering to a strict Three-Tier Logical Architecture:

TIER 1: PRESENTATION LAYER

React.js SPA · Vite ES-Module Bundler · Port 5173

Student Portal
/student/*

Counsellor Portal
/counsellor/*

Admin Dashboard
/admin/*

Axios HTTP/REST

TIER 2: APPLICATION LAYER

Node.js + Express.js API Server · Port 5000

authRoutes screeningRts counsellorRts adminRoutes

CORS → Body Parser → express-session → Passport.js → Joi
riskEngine.js · notifController.js

TIER 3: PERSISTENCE LAYER

MySQL 8.x · InnoDB Engine · smhms_db

users screening_records mood_logs risk_profiles
counseling_sessions notification_logs consent_records
audit_logs student_acad_data pattern_reports

3.2 Technology Stack

lightgray Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Single-Page Application with declarative UI rendering

lightgray Layer	Technology	Version	Purpose
Bundler	Vite	7+	Native ES-Module development server with HMR
HTTP Client	Axios	Latest	Promise-based HTTP client for API communication
Backend Runtime	Node.js	18+	Asynchronous, non-blocking event-loop server
Web Framework	Express.js	4.x	Middleware-driven HTTP request/response pipeline
Authentication	Passport.js + passport-local	Latest	Email/bcrypt credential verification with optional 2FA
Input Validation	Joi	Latest	Schema-based request body validation (PHQ-9, GAD-7, PSS ranges)
Session Management	express-session	Latest	Server-side session persistence with 24-hour TTL
Encryption	Node.js crypto (AES-256-CBC)	Built-in	Session note encryption using PBKDF2-derived key
Database	MySQL	8.x	ACID-compliant relational database with InnoDB engine
DB Driver	mysql2/promise	Latest	Async/await-compatible MySQL driver with connection pooling
Notifications	Nodemailer	Latest	Email delivery for High-risk alerts and session reminders
Secrets	dotenv	Latest	Environment variable injection

3.3 Database Schema — Entity Relationship Summary

The `smhms_db` database comprises ten interrelated tables enforcing strict referential integrity via InnoDB foreign keys:

`users` (PK: `id` UUID, `role` ENUM, `locked`, `failed_attempts`)

`screening_records` (`student_id` FK → `users.id`, CASCADE)
`phq9_score`, `gad7_score`, `pss_score`, `risk_tier`, `is_complete`

`mood_logs` (`student_id` FK → `users.id`, CASCADE)
 UNIQUE(`student_id`, `log_date`)

`risk_profiles` (`student_id` UNIQUE FK → `users.id`, CASCADE)
`composite_score` FLOAT, `risk_tier` ENUM, `alert_sent` BOOL

`counseling_sessions` (`student_id` FK, `counsellor_id` FK → `users.id`)
`status` ENUM('Requested', 'Confirmed', 'Completed', 'Cancelled')
`session_notes` TEXT (AES-256 ciphertext)

`notification_logs` (`recipient_id` FK → `users.id`, CASCADE)

```

    retry_count INT, ack_at DATETIME

consent_records    (student_id UNIQUE FK, guardian_id FK)
    revoked_at nullable DATETIME

student_acad_data  (student_id FK → users.id, CASCADE)
    grades JSON, engagement_score FLOAT

audit_logs         (actor_id FK → users.id, SET NULL on delete)
    hash_chain VARCHAR(64) ← SHA-256 tamper-evident trigger

```

Key Schema Constraints

lightgray Constraint	Table	Column	Effect
PRIMARY KEY UUID	All tables	id	Unique row identification
UNIQUE	users	email	Prevents duplicate accounts
UNIQUE	mood_logs	(student_id, log_date)	Enforces one log per student per day
UNIQUE	risk_profiles	student_id	One active risk profile per student
FOREIGN KEY CASCADE	All student tables	student_id	Referential integrity; orphans rejected
CHECK	screening_records	phq9_score	Must be BETWEEN 0 AND 27
ENUM	counseling_sessions	status	Valid states only: Requested/Confirmed/Completed/Cancelled
BEFORE INSERT Trigger	audit_logs	hash_chain	SHA-256(prev_hash new_row) tamperevidence
DEFAULT 'Requested'	counseling_sessions	status	All new sessions begin in Requested state

3.4 Subsystem Decomposition

lightgray Subsystem	Key Source Files	Core Responsibility	In Scope
Authentication & Access Control	authRoutes.js, server.js (Passport strategy)	Email/bcrypt login, lockout, session lifecycle, RBAC middleware	Yes

lightgray Subsystem	Key Source Files	Core Responsibility	In Scope
Screening & Risk Assessment	screeningRoutes.js, riskEngine.js	PHQ-9/GAD-7/PSS submission, risk scoring, profile UPSERT, High-risk alert	Yes
Mood Logging & Profile Management	moodRoutes.js, riskRoutes.js	Daily mood log with uniqueness guard, mood history, risk profile fetch	Yes
Counselling Session & Audit Management	counsellorRoutes.js, adminRoutes.js, audit_logs table	Session state transitions, encrypted notes, audit trail with hash chain	Yes
Pattern Analysis & ML Reporting	analyticsRoutes.js	Pattern report generation via ML model; requires 4 weeks data	Out of Scope
Guardian Consent Portal	consentRoutes.js	Guardian data access, consent grant/revoke workflow	Out of Scope

3.5 Counselling Session State Machine

The `counseling_sessions.status` field implements a deterministic finite state machine. Valid state transitions are:

```
Session      Requested      (DEFAULT on INSERT)
Created
```

```
    Counsellor confirms via
    PUT .../confirm
```

```
    Confirmed
```

```
    Session conducted;
    PUT .../complete
```

```
    Completed
```

```
At any pre-Completed stage:
PUT .../cancel  Cancelled (terminal state)
```

Invariant: Every state transition atomically triggers an INSERT into `audit_logs` with `actor_id` = the acting counsellor or student, and the `hash_chain` trigger fires automatically. This is verified in Subsystem 4 test cases.

4. Features to be Tested / Not to be Tested

4.1 Features to be Tested

4.1.1 Subsystem 1 — Authentication & Access Control

lightgray Feature ID	Feature Description	Level	Technique
AUTH-F01	POST /api/auth/login with valid credentials returns session + role	Unit	White-Box
AUTH-F02	POST /api/auth/login with invalid password returns 401	Unit	White-Box
AUTH-F03	Account locks after 5 consecutive failed login attempts	Integration	White-Box
AUTH-F04	Locked account returns 423 on subsequent login attempt	Unit	White-Box
AUTH-F05	GET /api/auth/me returns {id, role, email} for authenticated session	Unit	White-Box
AUTH-F06	GET /api/auth/me returns 401 for unauthenticated request	Unit	White-Box
AUTH-F07	POST /api/auth/logout destroys session; subsequent /auth/me returns 401	Integration	White-Box
AUTH-F08	RBAC: Student session cannot access /api/counsellor/* or /api/admin/*	Integration	White-Box
AUTH-F09	RBAC: Counsellor session cannot access /api/admin/* endpoints	Integration	White-Box
AUTH-F10	Student login renders Student Portal dashboard in browser	System	Black-Box

4.1.2 Subsystem 2 — Screening & Risk Assessment

lightgray Feature ID	Feature Description	Level	Technique
SCRN-F01	Joi rejects PHQ-9 score outside 0–27 range with 422	Unit	White-Box
SCRN-F02	Joi rejects GAD-7 score outside 0–21 range with 422	Unit	White-Box
SCRN-F03	Joi rejects PSS score outside 0–40 range with 422	Unit	White-Box
SCRN-F04	Valid submission inserts row in <code>screening_records</code> with <code>is_complete = 1</code>	Integration	White-Box
SCRN-F05	<code>riskEngine.score()</code> computes correct composite score and tier	Unit	White-Box
SCRN-F06	<code>risk_profiles</code> UPSERT occurs atomically within same transaction as screening INSERT	Integration	White-Box
SCRN-F07	High-risk submission (composite 70) triggers counsellor notification	Integration	White-Box
SCRN-F08	GET <code>/api/student/screenings</code> returns ordered screening history	Unit	White-Box
SCRN-F09	GET <code>/api/student/risk</code> returns current risk tier and composite score	Unit	White-Box
SCRN-F10	Screening form UI accepts and submits valid scores; confirms risk tier to student	System	Black-Box
SCRN-F11	Counsellor at-risk dashboard shows students with Medium/High risk tier	System	Black-Box

4.1.3 Subsystem 3 — Mood Logging & Profile Management

lightgray Feature ID	Feature Description	Level	Technique
MOOD-F01	POST /api/student/moodlogs inserts row with correct student_id and log_date	Unit	White-Box
MOOD-F02	UNIQUE constraint (student_id, log_date) rejects duplicate same-day submission with 409	Unit	White-Box
MOOD-F03	Mood score must be 1–10; Joi rejects out-of-range with 422	Unit	White-Box
MOOD-F04	Sleep hours must be 0–24; DB CHECK constraint rejects violations	Unit	White-Box
MOOD-F05	GET /api/student/moodlogs returns all logs ordered by log_date DESC	Unit	White-Box
MOOD-F06	GET /api/student/risk returns composite_score, risk_tier, classified_at	Unit	White-Box
MOOD-F07	Student Profile page renders name, email, and risk tier from backend	System	Black-Box
MOOD-F08	Mood log UI allows submission; duplicate-day error message shown correctly	System	Black-Box

4.1.4 Subsystem 4 — Counselling Session & Audit Management

lightgray Feature ID	Feature Description	Level	Technique
CSLN-F01	POST /api/student/sessions creates row with status = 'Requested'	Unit	White-Box

lightgray Feature ID	Feature Description	Level	Technique
CSLN-F02	PUT /api/counsellor/sessions/:id/confirm updates status to 'Confirmed' from 'Requested'	Unit	White-Box
CSLN-F03	State-guard middleware rejects confirm when status is not 'Requested' with 409	Unit	White-Box
CSLN-F04	PUT .../complete updates status to 'Completed' from 'Confirmed'	Unit	White-Box
CSLN-F05	Session notes stored as AES-256-CBC ciphertext; plaintext absent from DB	Integration	White-Box
CSLN-F06	Confirm action atomically inserts audit_logs row with correct actor_id	Integration	White-Box
CSLN-F07	audit_logs hash_chain computed by BEFORE INSERT trigger (SHA-256)	Unit	White-Box
CSLN-F08	GET /api/counsellor/sessions returns all sessions assigned to counsellor	Integration	White-Box
CSLN-F09	GET /api/admin/auditlogs returns all audit entries ordered by ts DESC	Integration	White-Box
CSLN-F10	Counsellor portal displays pending session queue; Confirm button changes status	System	Black-Box
CSLN-F11	Admin audit log view shows all events with actor type and timestamp	System	Black-Box

4.2 Features Not to be Tested

lightgray Feature	Rationale
Pattern Analysis & ML Reporting (/api/analytics/*)	Requires 4 weeks of data; outside the four targeted subsystems
Guardian Consent Portal (/api/student/consent)	Standalone privacy feature, not mapped to tested subsystems
Admin user management CRUD	Not among the four selected subsystems
Two-Factor Authentication (TOTP)	twofa_enabled flag is set but passport-totp integration is deferred
Email delivery internals (Nodemailer SMTP)	Owned by third-party; only the triggering logic is tested
Cross-browser compatibility	Testing conducted on Google Chrome only
Performance / load testing	Outside this functional test plan's scope
Mobile responsiveness	CSS media query behaviour not evaluated

5. Pass/Fail Criteria

5.1 Individual Test Case Criteria

lightgray Verdict	Condition
PASS	Actual output exactly matches the documented Expected Result — correct HTTP status code, response body structure, and database state all verified.
FAIL	Actual output deviates from Expected Result in any dimension — wrong status code, malformed body, missing database row, UI error, or unexpected exception.

5.2 Subsystem-Level Pass Criteria

A subsystem is considered passed when all of the following conditions are met:

lightgray Criterion	Threshold
Critical test pass rate	100% — all auth, data integrity, encryption, and state-transition tests must pass
Overall test pass rate	90% of all tests within the subsystem
Severity-1 defects open	0 — no crashes, data corruption, plaintext exposure, or security breaches permitted
Database integrity	100% — no orphaned records or FK/UNIQUE constraint violations
Audit log integrity	100% — every state-transition test must produce a verifiable hash-chain entry

5.3 Overall Plan Pass Criteria

The test plan is considered successfully executed when:

1. All four subsystems meet the criteria defined in Section 5.2.
2. The end-to-end flow completes without failure: Student registers → submits screening → High-risk alert generated → Counsellor confirms session → Student views Confirmed status → Audit log entry verified.
3. Session notes in `counseling_sessions` table contain ciphertext, not plaintext.
4. All `[INSERT SCREENSHOT]` placeholders are fulfilled with captured evidence.

5.4 Defect Severity Classification

lightgray Severity	Definition	Example
1 — Critical	System crash, data loss, plaintext mental-health data exposed, or security vulnerability. Testing halts immediately.	Bcrypt bypass allows login without password; session notes stored in plaintext.
2 — Major	Core feature non-functional; system remains operational.	Screening submission returns success but no DB row is created; risk profile not updated.
3 — Minor	Feature works with non-critical deviations.	Timestamp format differs from expected; risk tier label capitalisation inconsistent.
4 — Trivial	Cosmetic issue with no functional impact.	Misaligned padding on a dashboard card.

6. Approach

6.1 Strategy Overview

The testing approach combines Black-Box and White-Box techniques across three progressive levels:

SYSTEM TESTING (Black-Box)

End-to-end browser-based workflows

INTEGRATION TESTING (White-Box)

API Database Session riskEngine

UNIT TESTING (White-Box)

Individual route handlers, SQL queries, Joi validators, riskEngine

6.2 Black-Box Techniques

lightgray Technique	Application
Equivalence Partitioning	Divides inputs into valid/invalid classes. Example: Valid PHQ-9 = 0–27; Invalid = –1 or 28.
Boundary Value Analysis	Tests at constraint edges. Example: PHQ-9 = 0 (minimum), PHQ-9 = 27 (maximum), PHQ-9 = 28 (just over). Mood score = 1 (min), 10 (max).
State Transition Testing	Validates FSM: Requested → Confirmed → Completed are valid; Completed → Confirmed is invalid.
Use Case Testing	Full scenario walkthroughs. Example: Login → Submit Screening → View Risk Tier → Request Session → Track Status.

Tools: Google Chrome, React dev server (<http://localhost:5173>).

6.3 White-Box Techniques

lightgray Technique	Application
Statement Coverage	Every line in route handlers is exercised. Example: <code>if (user.locked)</code> branch triggered by failed-5-times account.
Branch Coverage	Both true/false outcomes of conditionals. Example: <code>if (!req.user)</code> in <code>/auth/me</code> — tested with and without active session.
Path Coverage	Full middleware pipeline traced: CORS → Body Parser → Session → Passport → Joi → Route Handler → MySQL → Response.
Data Flow Testing	Variable traces from definition to use. Example: <code>phq9Score</code> flows from request body → Joi validation → <code>riskEngine.score()</code> → <code>risk_profiles</code> UPSERT.

Tools: Postman (API testing), MySQL Workbench (database verification).

6.4 Testing Levels

lightgray Level	Scope	Method	Verification
Unit	Individual route handlers, Joi validators, <code>riskEngine.js</code> , middleware guards	Direct Postman API calls with controlled payloads	HTTP status + response body + DB state

Integration	API Database, Session RBAC, riskEngine notification, Transaction atomicity	Sequential API calls; DB state checked at each step	Cross-table end-state correctness
System	Complete end-to-end user workflows through browser UI	Manual browser interaction following prescribed steps	Visual confirmation + data consistency across portals

6.5 Test Environment

lightgray Component	Configuration
OS	macOS / Windows 10+
Browser	Google Chrome (Latest Stable)
Frontend	npm run dev → http://localhost:5173
Backend	node server.js → http://localhost:5000
Database	MySQL 8.x, smhms_db, seeded via seed-database.sql
API Tool	Postman (Latest)
DB Client	MySQL Workbench or MySQL CLI

6.6 Seed Test Data

lightgray Entity	Record	Key Fields
Admin	System Admin	admin@smhms.edu, role='Admin'
Counsellor	Dr. Priya Mehta	counsellor@smhms.edu, role='Counsellor'
Student	Arjun Sharma	arjun@student.vjti.ac.in, role='Student', locked=0, failed_attempts=0
Student (locked)	Test Locked	locked@student.vjti.ac.in, locked=1, failed_attempts=5
Screening Record	Sample PHQ-9	phq9=20, gad7=15, pss=30 → High-risk tier
Mood Log	Day 1	mood_score=3, sleep_hours=5.0, log_date=today

Important: Re-run `seed-database.sql` before each test batch to restore a deterministic baseline state.

6.7 Execution Methodology

For each test case:

1. **Setup** — Verify pre-conditions (servers running, DB seeded, browser cookies cleared).

2. **Execute** — Follow documented steps sequentially and exactly as written.
3. **Observe** — Compare actual result against Expected Result.
4. **Capture** — Screenshot at every **[INSERT SCREENSHOT: ...]** placeholder.
5. **Record** — Mark PASS or FAIL in the results column.

7. Suspension and Resumption Criteria

7.1 Suspension Criteria

Testing shall be immediately suspended upon any of the following:

lightgray ID	Condition	Severity	Action
SC-01	Backend server at <code>http://localhost:5000</code> becomes unresponsive	Critical	Restart via <code>node server.js</code> ; verify DB connectivity
SC-02	MySQL <code>smhms_db</code> unreachable or connection pool exhausted	Critical	Restart MySQL daemon; re-seed if needed
SC-03	Session notes found in plaintext in the database	Critical	Halt immediately; log Severity-1 defect; check AES key derivation
SC-04	Seed data corrupted — DB deviates from expected baseline	Major	Re-execute <code>seed-database.sql</code>
SC-05	Severity-1 defect discovered (crash, data loss, plaintext exposure, auth bypass)	Critical	Log defect with reproduction steps; halt all testing until resolved
SC-06	Vite dev server fails to compile	Major	Fix compilation errors; restart with <code>npm run dev</code> ; API tests may continue

7.2 Resumption Criteria

Testing resumes only when all of the following are confirmed:

1. Frontend (`http://localhost:5173`) and backend (`http://localhost:5000`) are operational.
2. GET `/api/health` returns `{"status":"success","message":"Backend running"}`.
3. Database re-seeded and all 10 tables verified with correct row counts.
4. Session notes column in `counseling_sessions` confirmed to store ciphertext.
5. Any Severity-1 defects resolved and verified.

6. All browser sessions and cookies cleared.

8. Testing Materials (Hardware/Software Requirements)

8.1 Hardware Requirements

lightgray Component	Minimum	Recommended
Processor	Intel Core i3 / Apple M1	Intel Core i5+ / Apple M1 Pro+
RAM	4 GB	8 GB+
Storage	500 MB free	1 GB+ free
Network	Localhost loopback	Active internet (for Nodemailer SMTP)
Display	1280×720	1920×1080

8.2 Software Requirements

lightgray Software	Version	Purpose
Node.js	18.x+	Backend runtime
npm	9.x+	Package manager
MySQL Server	8.x	Database
MySQL Workbench (or CLI)	Latest	DB inspection and query execution
Google Chrome	Latest Stable	Black-Box system testing
Postman	Latest	White-Box API testing

8.3 Environment Setup Commands

1. Seed the database

```
mysql -u root -p < seed-database.sql
```

2. Start backend

```
cd backend && node server.js
```

Expected: "Database connected" + "SMHMS Server running on http://localhost:5000"

3. Start frontend

```
cd frontend && npm run dev
```

Expected: "Local: http://localhost:5173"

4. Verify health

```
curl http://localhost:5000/api/health
```

Expected: {"status": "success", "message": "Backend running"}

9. Test Cases

Notation: Each table row is one test case. Screenshot placeholders appear in bold below the relevant row and must be fulfilled with captured evidence during execution.

9.1 Subsystem 1: Authentication & Access Control

Unit Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/Fail
AUTH-UT-01	POST /api/auth/login returns session and role for valid student credentials	Backend + DB running; Arjun's account seeded	1. Open Postman. 2. POST /api/auth/login with {"email": "arjun@...", "password": "..."}. 3. Inspect response and cookies.	200 OK; {"role": "Student"}; connect.sid cookie set.	
AUTH-UT-02	Invalid password returns 401	Backend running; valid user exists	1. POST /api/auth/login with wrong password. 2. Inspect response.	401 Unauthorized; {"message": "Invalid credentials"}. failed_attempts incremented in DB.	
AUTH-UT-03	Locked account returns 423	locked=1 record seeded	1. POST /api/auth/login with locked account credentials.	423 Locked; {"message": "Account locked"}.	
AUTH-UT-04	GET /api/auth/me returns user object for authenticated session	Backend running; valid session exists	1. GET http://localhost:5001/api/auth/me with session cookie. 2. Inspect response.	200 OK; body: {"id": "api/auth/me", "role": "Student", "email": "arjun@..."} matching logged-in user.	
AUTH-UT-05	GET /api/auth/me returns 401 for unauthenticated request	Backend running; no cookies	1. New Postman session (no cookies). 2. GET /api/auth/me.	401 Unauthorized; {"message": "Not logged in"}.	

[INSERT SCREENSHOT: Postman showing 200 OK with role:Student on successful login]

[INSERT SCREENSHOT: Postman showing 401 Unauthorized on /api/auth/me without session cookie]

Integration Tests (White-Box)

Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/Fail
AUTH-IT-01	Account locks after 5th failed login; 6th attempt returns 423	Arjun's account exists; failed_attempts	1. POST /api/auth/login with wrong password 5 times. 2. SQL: SELECT locked FROM users WHERE email=?. 3. POST again.	After 5 attempts: locked=1 in DB. 6th attempt: 423 Locked.	
AUTH-IT-02	Logout destroys session; subsequent /auth/me returns 401	User authenticated	1. POST /api/auth/logout with session cookie. 2. GET /api/auth/me with same cookie.	Logout: 200 OK, {"success":true}. Subsequent /auth/me: 401.	
AUTH-IT-03	RBAC: Student session cannot access counsellor endpoint	Student session active	1. Login as Student. 2. GET /api/counsellor/admins with student cookie. 3. Inspect response.	403 Forbidden or 401 Unauthorized. Counsellor-privileged data not returned.	
AUTH-IT-04	RBAC: Counsellor session cannot access admin endpoint	Counsellor session active	1. Login as Counsellor. 2. GET /api/admin/users with counsellor cookie. 3. Inspect response.	403 Forbidden. Admin user list not returned.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
AUTH-IT-05	Session cookie persists across requests with 24-hour TTL	User authenticated	1. GET /api/auth/me — note userId. 2. Wait 5 seconds. 3. GET /api/auth/me again.	Both calls return identical user objects. Cookie maxAge set to 86400000 ms.	

[INSERT SCREENSHOT: Postman showing 423 Locked on 6th failed login attempt; MySQL Workbench showing locked=1]

[INSERT SCREENSHOT: Postman showing 403 Forbidden on student accessing /api/counsellor/atrisk]

System Tests (Black-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
AUTH-ST-01	End-to-end login flow for Student	Frontend + Backend running; browser cookies cleared	1. Open http://localhost:3000 2. Enter student credentials. 3. Click Login. 4. Wait for redirect.	Student Dashboard loads with student's name. No errors in console.	
AUTH-ST-02	Counsellor login redirects to Counsellor Portal	Counsellor account seeded	1. Login as Counsellor. 2. Observe redirect URL.	Redirects to /counsellor/dashboard. Session role = 'Counsellor'.	
AUTH-ST-03	End-to-end logout flow	User logged into dashboard	1. Click "Logout" button. 2. Observe redirect. 3. Attempt to navigate back to dashboard URL.	Redirected to login page. Direct dashboard URL access denied.	

[INSERT SCREENSHOT: Student Dashboard showing logged-in student's name after successful login]

9.2 Subsystem 2: Screening & Risk Assessment

Unit Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/Fail
SCRN-UT-01	Joi rejects PHQ-9 = 28 with 422	Backend running; student session active	1. POST /api/student/screening with {phq9Score:28, gad7Score:10, pssScore:20}.	422 Unprocessable Entity; error message referencing phq9Score must be 27.	
SCRN-UT-02	Joi rejects GAD-7 = 22 with 422	Backend running; student session active	1. POST /api/student/screening with {phq9Score:10, gad7Score:22, pssScore:20}.	422 Unprocessable Entity; error message referencing gad7Score must be 21.	
SCRN-UT-03	Joi rejects PSS = -1 with 422	Backend running; student session active	1. POST /api/student/screening with {phq9Score:10, gad7Score:10, pssScore:-1}.	422 Unprocessable Entity; error message referencing pssScore must be 0.	
SCRN-UT-04	riskEngine.score() computes High tier for severe inputs	riskEngine.js loaded	1. Call riskEngine.score({phq9Score:25, gad7Score:20, pssScore:38, moodTrend:2, acadScore:0.1}). 2. Inspect returned {score, tier}.	tier='High'; composite score 70.	
SCRN-UT-05	riskEngine.score() computes Low tier for minimal inputs	riskEngine.js loaded	1. Call riskEngine.score({phq9Score:2, gad7Score:1, pssScore:5, moodTrend:9, acadScore:0.9}).	tier='Low'; composite score 40.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
SCRN-UT-06	GET /api/student/screenings returns ordered history	Student has 2 screening records	1. GET http://localhost:8000/api/student/screenings with student cookie. 2. Inspect array.	200 OK; array ordered by submitted_at DESC; each object contains phq9_score, gad7_score, pss_score, risk_tier.	

[INSERT SCREENSHOT: Postman showing 422 with Joi validation error for PHQ-9 out-of-range]

Integration Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
SCRN-IT-01	Valid submission inserts screening_records row and UPSERTs risk_profiles atomically	Backend + DB running; student session active; re-seed DB	1. Note COUNT(*) in both tables. 2. POST /api/student/screenings with valid scores. 3. SQL: SELECT * FROM screening_records ORDER BY id DESC LIMIT 1. 4. SQL: SELECT * FROM risk_profiles WHERE student_id=?.	201 Created; {recordId, riskTier}. New row in screening_records. risk_profiles row updated/created with same tier. Both counts incremented.	
SCRN-IT-02	Transaction rolls back if risk engine fails; no partial state	Backend + DB; riskEngine temporarily broken (mock)	1. POST valid screening while riskEngine throws. 2. SQL: Verify no new row in screening_records.	500 Internal Server Error. No new row in screening_records — transaction fully rolled back.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
SCRN-IT-03	High-risk submission triggers notification insert	Student session; notification_logs count noted	1. POST /api/student/screening with phq9=25, gad7=20, pss=38. 2. SQL: SELECT * FROM notification_logs ORDER BY id DESC LIMIT 1.	201 Created; riskTier='High'. New row in notification_logs with counsellor as recipient_id.	
SCRN-IT-04	Boundary value: PHQ-9 = 27 (max valid) accepted	Student session active	1. POST /api/student/screening with phq9Score:27, gad7Score:0, pssScore:0.	201 Created. Row inserted with phq9_score=27. No validation error.	

[INSERT SCREENSHOT: Postman 201 Created on valid screening + MySQL Workbench showing new row in screening_records and updated risk_profiles]

System Tests (Black-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
SCRN-ST-01	Student submits screening form and sees risk tier on UI	Student logged in; Frontend running	1. Navigate to Screening page. 2. Enter valid PHQ-9, GAD-7, PSS scores. 3. Submit. 4. Observe result.	Success message displayed; risk tier (Low/Medium/High) shown to student. Page does not crash.	
SCRN-ST-02	Counsellor at-risk list displays High-risk students	Counsellor logged in; at least one High-risk student exists	1. Login as Counsellor. 2. Navigate to At-Risk dashboard. 3. Observe list.	High-risk students displayed with name, composite score, and classified date. List ordered by score descending.	

[INSERT SCREENSHOT: Student screening form UI showing risk tier result after submission]

[INSERT SCREENSHOT: Counsellor at-risk dashboard showing High/Medium risk students]

9.3 Subsystem 3: Mood Logging & Profile Management

Unit Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/Fail
MOOD-UT-01	Valid mood log inserted with correct student_id and today's date	Student session; no mood log for today	1. POST /api/student/moodlogs with {mood_score:6, sleep_hours:7.5, stressors:"exams"} 2. SQL: SELECT * FROM mood_logs ORDER BY id DESC LIMIT 1.	201 Created. Row has correct student_id, log_date = CURDATE(), mood_score=6, sleep_hours=7.5.	
MOOD-UT-02	Duplicate same-day submission returns 409	Mood log for today already exists for student	1. POST /api/student/moodlogs again on same day.	409 Conflict; {message: "Mood log already submitted for today"}. DB row count unchanged.	
MOOD-UT-03	Joi rejects mood_score = 11 with 422	Student session active	1. POST /api/student/moodlogs with mood_score:11.	422 Unprocessable Entity; Joi error referencing mood_score max = 10.	
MOOD-UT-04	Joi rejects sleep_hours = 25 with 422	Student session active	1. POST /api/student/moodlogs with sleep_hours:25.	422 Unprocessable Entity; Joi error referencing sleep_hours max = 24.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
MOOD-UT-05	GET /api/student/moodlogs returns history ordered by date	Student has 3 mood logs across different dates	1. GET http://localhost:5000/api/student/moodlogs with student cookie.	200 OK; array ordered by log_date DESC; each object has mood_score, sleep_hours, stressors, log_date.	
MOOD-UT-06	GET /api/student/risk returns risk profile fields	Risk profile exists for student	1. GET http://localhost:5000/api/student/risk with student cookie.	200 OK; {mood_tier, risk_tier, classified_at, alert_sent} returned.	

[INSERT SCREENSHOT: Postman showing 409 Conflict on duplicate same-day mood log submission]

[INSERT SCREENSHOT: MySQL Workbench showing UNIQUE constraint on (student_id, log_date)]

Integration Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
MOOD-IT-01	UNIQUE constraint at DB level rejects duplicate (verified via direct SQL)	Today's mood log exists for Arjun	1. MySQL Workbench: INSERT INTO mood_logs (student_id, mood_score, sleep_hours, log_date) VALUES (?, 5, 6, CURDATE()).	ERROR 1062: Duplicate entry for key uq_one_per_day. Insert rejected at DB level.	
MOOD-IT-02	Foreign key rejects mood log for non-existent student	No user with UUID '00000000-0000-0000-0000-000000000000'	1. SQL: INSERT INTO mood_logs (...) VALUES ('00000000-...', 5, 7, CURDATE()).	ERROR 1452: Cannot add or update child row — FK constraint violated. Insert rejected.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
MOOD-IT-03	Risk profile fetch returns correct joined data after screening	Student has screening record and risk profile	1. GET /api/student/risk 2. Compare composite_score against manually computed value.	200 OK; score matches expected computation from seeded screening values.	

System Tests (Black-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
MOOD-ST-01	Mood log UI allows submission and shows success	Student logged in; no log for today	1. Navigate to Mood Log page. 2. Enter mood score and sleep hours. 3. Submit.	Success confirmation displayed. New log visible in mood history.	
MOOD-ST-02	Second same-day submission shows error on UI	Student logged in; today's log already submitted	1. Navigate to Mood Log page. 2. Attempt to submit again.	Error message displayed (e.g., "You have already logged your mood today").	

[INSERT SCREENSHOT: Student Mood Log page showing success confirmation and history list]

9.4 Subsystem 4: Counselling Session & Audit Management

Unit Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/Fail
CSLN-UT-01	POST /api/student/sessions creates row with status='Requested'	Student session; counsellor ID known	1. POST /api/student/sessions with {counsellorId, requestedDate}. 2. SQL: SELECT status FROM counseling_sessions ORDER BY id DESC LIMIT 1.	201 Created. DB row: status='Requested', correct student_id.	
CSLN-UT-02	PUT .../confirm updates status to 'Confirmed'	Session exists with status='Requested', counsellor session	1. PUT http://localhost:5000/api/counsellor/sessions/ 2. SQL: SELECT status FROM counseling_sessions WHERE id=?.	200 OK; DB: status='Confirmed'.	
CSLN-UT-03	State-guard rejects confirm when status is not 'Requested'	Session already status='Confirmed'	1. PUT .../confirm again on same session.	409 Conflict; {"message": "Session is not in Requested state"}. Status unchanged in DB.	
CSLN-UT-04	PUT .../complete updates status to 'Completed'	Session status='Confirmed'	1. PUT /api/counsellor/sessions/ 2. SQL: SELECT status FROM counseling_sessions WHERE id=?.	200 OK. DB: status='Completed'.	
CSLN-UT-05	Audit log hash chain trigger fires on session confirm	audit_logs row count noted; session Confirmed via CSLN-UT-02	1. SQL: SELECT * FROM audit_logs ORDER BY id DESC LIMIT 1. 2. Verify hash_chain is non-empty 64-char hex string.	New row: action_type='DataAccess' or 'ConfigChange' (appropriate type); hash_chain is non-empty. Count incremented by 1.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
CSLN-UT-06	Session notes stored as ciphertext, not plaintext	Session notes added via PUT .../notes	1. PUT /api/counsellor/sessions/notes with {"notes": "Student is anxious"}. 2. SQL: SELECT session_notes FROM counseling_sessions WHERE id=?.	DB session_notes column contains ciphertext (not readable text). String does NOT contain "anxious" or any plaintext word from the input.	

[INSERT SCREENSHOT: Postman showing 409 Conflict on invalid state transition (confirm on already-confirmed session)]

[INSERT SCREENSHOT: MySQL Workbench showing session_notes as ciphertext (not plaintext)]

[INSERT SCREENSHOT: MySQL Workbench showing audit_logs row with non-empty hash_chain after session confirm]

Integration Tests (White-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
CSLN-IT-01	Counsellor queue returns all Requested sessions with joined student data	At least one status='Requested' session exists	1. GET http://localhost:8000/api/counsellor/sessions with counsellor cookie. 2. Inspect array.	200 OK; array contains objects with student_name, student_email, risk_tier, requested_date, status='Requested'. Correctly joined from users and risk_profiles.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/Fail
CSLN-IT-02	Confirmed session disappears from Requested-only view after confirm	Session ID noted in queue	1. GET /api/counsellor/sessions (Requested filter). 2. PUT confirm. 3. GET again.	After confirm, session absent from Requested queue. WHERE status='Requested' filter correctly applied.	
CSLN-IT-03	Hash chain is sequential: second audit entry's hash includes first entry's hash	At least one entry in audit_logs	1. Trigger two state changes (request + confirm). 2. SQL: SELECT id, hash_chain FROM audit_logs ORDER BY id DESC LIMIT 2.	Second entry's hash_chain = SHA2(concat(first_hash, new_row_data), 256). Chain is computationally linked.	
CSLN-IT-04	End-to-end state propagation: Student sees Confirmed status after counsellor confirms	Session created by student; counsellor confirms	1. PUT confirm. 2. GET /api/student/sessions with student cookie. 3. Find session by ID.	Student endpoint returns status='Confirmed' for the same session. Single source of truth in counseling_sessions.	
CSLN-IT-05	GET /api/admin/auditlogs returns all events with correct structure	Admin session; at least 2 audit entries exist	1. GET http://localhost:3000/api/admin/auditlogs with admin cookie. 2. Inspect response.	200 OK; array of {actor_id, action_type, target_type, target_id, ts, hash_chain} objects; ordered by ts DESC.	

[INSERT SCREENSHOT: Postman showing GET /api/counsellor/sessions with joined student data and Requested status]

[INSERT SCREENSHOT: MySQL Workbench showing two sequential audit_log rows with linked hash_chain values]

System Tests (Black-Box)

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
CSLN-ST-01	Counsellor views pending session queue on dashboard	Counsellor logged in; at least one Requested session	1. Login as Counsellor. 2. Navigate to Sessions dashboard. 3. Observe queue.	Dashboard displays pending sessions with student name, risk tier, and requested date. Each row has Confirm/Cancel buttons.	
CSLN-ST-02	Counsellor confirms session; status updates in Student portal	Counsellor logged in; Requested session visible	1. Click "Confirm" on a session. 2. Observe queue. 3. Login as corresponding student. 4. Navigate to My Sessions.	Session disappears from Requested queue. In Student portal, same session shows status='Confirmed'.	
CSLN-ST-03	Admin views complete audit log with all events	Admin logged in; at least one confirm/complete action recorded	1. Login as Admin. 2. Navigate to Audit Log section. 3. Observe entries.	Audit log table displays entries with Actor Type, Action Details, and Action Time. Session confirmation and completion entries visible with correct timestamps.	

lightgray Test ID	Description	Pre-conditions	Steps	Expected Result	Pass/ Fail
CSLN-ST-04	End-to-end mental health workflow: screening → alert → session → completion	Student, Counsellor, and Admin accounts seeded	1. Login as Student; submit High-risk screening. 2. Login as Counsellor; verify alert; confirm session. 3. Mark session complete. 4. Login as Admin; verify audit trail.	High-risk screening creates notification. Session transitions Requested → Confirmed → Completed. Admin audit log shows all three events in correct order with valid hash chain.	

[INSERT SCREENSHOT: Counsellor portal showing session queue with student name, risk tier, and Confirm button]

[INSERT SCREENSHOT: Student portal showing session status = Confirmed after counsellor confirmation]

[INSERT SCREENSHOT: Admin audit log view showing all events with actor type, action details, and timestamps]

10. Testing Schedule

10.1 Overview

The testing activities are organised into four chronological phases mapped to the four targeted subsystems. This schedule assumes a two-person team executing manual tests on a configured local development environment. Each phase includes time for setup, execution, evidence capture (screenshots), and defect logging.

10.2 Detailed Schedule

lightgray Phase	Subsystem	Activities	Estimated Duration	Deliverables
Phase 0	All Subsystems	Environment Setup & Verification — seed database, start frontend and backend, verify <code>/api/health</code> , clear all browser cookies and sessions.	30 min	Verified environment; health check screenshot.
Phase 1	Authentication & Access Control	Unit Tests (AUTH-UT-01 to 05) — Postman API calls. Integration Tests (AUTH-IT-01 to 05) — logout, login, RBAC boundary tests. System Tests (AUTH-ST-01 to 03) — browser login/logout flows.	1.5 hrs	Completed test table with PASS/-FAIL for all 13 Auth test cases.
Phase 2	Screening & Risk Assessment	Unit Tests (SCRN-UT-01 to 06) — Joi validation, riskEngine, history. Integration Tests (SCRN-IT-01 to 04) — transaction atomicity, notification trigger, boundary values. System Tests (SCRN-ST-01 to 02) — UI screening and at-risk dashboard.	1.5 hrs	Completed test table for all 12 Screening test cases.
Phase 3	Mood Logging & Profile Management	Unit Tests (MOOD-UT-01 to 06) — uniqueness, validation, history, risk profile. Integration Tests (MOOD-IT-01 to 03) — DB constraints, FK rejection. System Tests (MOOD-ST-01 to 02) — UI log and duplicate error.	1 hr	Completed test table for all 11 Mood test cases.
Phase 4	Counselling Session & Audit Management	Unit Tests (CSLN-UT-01 to 06) — state transitions, encryption, hash chain. Integration Tests (CSLN-IT-01 to 05) — queue, chain integrity, propagation, admin view. System Tests (CSLN-ST-01 to 04) — full end-to-end workflow.	2 hrs	Completed test table for all 15 Counselling test cases.

lightgray Phase	Subsystem	Activities	Estimated Duration	Deliverables
Phase 5	All Subsystems	Defect Logging & Test Summary — compile PASS/FAIL results, log defects with severity, calculate pass rates, insert all captured screenshots into placeholders.	1 hr	Final test summary table; defect log (if any); complete Test Plan with evidence.

10.3 Testing Timeline (7-Day View)

lightgray Day	Activity
Day 1	Phase 0: Environment Setup + Phase 1: Authentication Testing
Day 2	Phase 2: Screening & Risk Assessment Testing
Day 3	Phase 3: Mood Logging & Profile Management Testing
Day 4	Phase 4: Counselling Session & Audit Management Testing
Day 5	Phase 5: Defect Logging, Evidence Compilation
Day 6	Screenshot insertion, document formatting, review
Day 7	Final submission preparation and PDF export

10.4 Test Case Summary

lightgray Subsystem	Unit Tests	Integration Tests	System Tests	Total
Authentication & Access Control	5	5	3	13
Screening & Risk Assessment	6	4	2	12
Mood Logging & Profile Management	6	3	2	11
Counselling Session & Audit Management	6	5	4	15
lightgray TOTAL	23	17	11	51

10.5 Testing Responsibilities

lightgray Role	Responsibility
----------------	----------------

Tester (Mokshi Shah)	Execute Subsystems 1 & 2 test cases; capture screenshot evidence; record PASS/FAIL verdicts; log defects.
Tester (Vanshika Shah)	Execute Subsystems 3 & 4 test cases; capture screenshot evidence; record PASS/FAIL verdicts; log defects.
Developer (Both Students)	Ensure test environment is correctly configured; resolve Severity-1/2 defects identified during testing.
Lab Evaluator (Instructor)	Review the completed Test Plan document with evidence against the grading rubric for Laboratory-10.

Complete Test Plan — Laboratory 10
Student Mental Health Monitoring System
— End of Document —